



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Lab experiment-11

Map Coloring using Constraint Satisfaction Problem (CSP)

Aim

To write a Python program to implement the Map Coloring Problem using the Constraint Satisfaction Problem (CSP) approach.

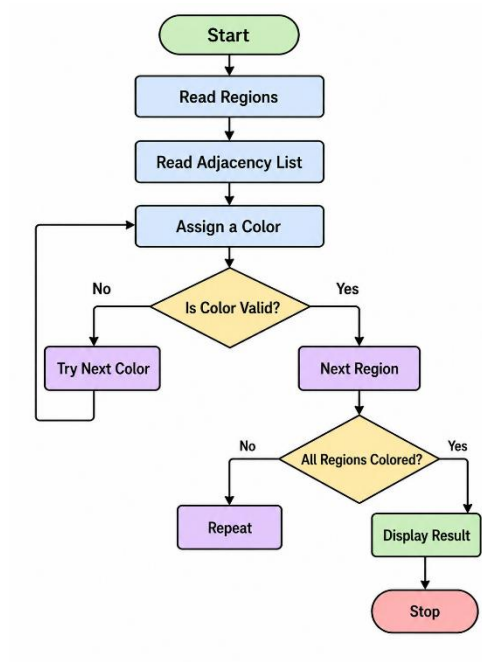
Objective

To assign colors to regions of a map such that no two adjacent regions have the same color.

Algorithm

1. Start.
2. Read the number of regions.
3. Read the names of the regions.
4. Read the adjacent regions.
5. Assign colors one by one.
6. Check whether adjacent regions have different colors.
7. If a valid assignment is found, display the coloring.
8. Stop.

Flowchart



Code

```
colors = ["Red", "Green", "Blue"]
```

```
n = int(input("Enter number of regions: "))
```

```
regions = []
```

```
graph = {}
```

```
for i in range(n):
```

```
    region = input(f"Enter region {i+1}: ")
```

```
    regions.append(region)
```

```
for region in regions:
```

```
    graph[region] = input(
```

```
        f"Enter neighbours of {region} (space-separated): "
    ).split()
```

```
assignment = {}
```

```
def is_safe(region, color):  
    for neighbour in graph[region]:  
        if assignment.get(neighbour) == color:  
            return False  
    return True
```

```
def solve(index):  
    if index == len(regions):  
        return True
```

```
    region = regions[index]
```

```
    for color in colors:  
        if is_safe(region, color):  
            assignment[region] = color  
            if solve(index + 1):  
                return True  
            del assignment[region]
```

```
    return False
```

```
if solve(0):  
    print("\nColor Assignment:")
```

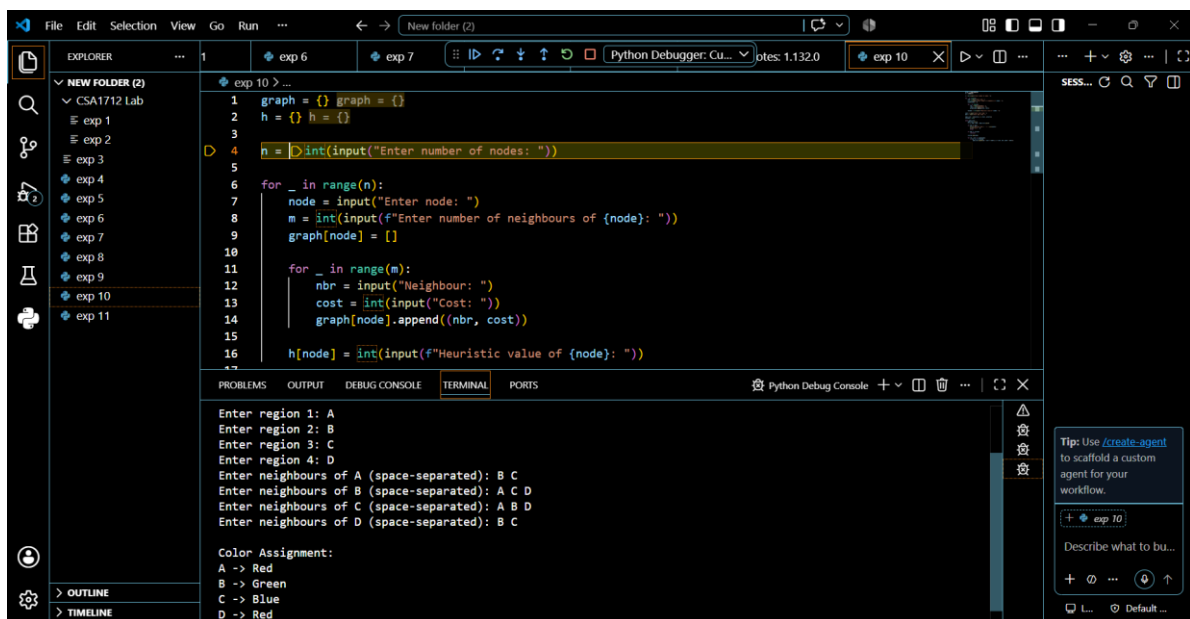
for region in regions:

```
print(region, "->", assignment[region])
```

else:

```
print("No solution exists.")
```

Output



```
1 graph = {}
2 h = {}
3
4 n = int(input("Enter number of nodes: "))
5
6 for _ in range(n):
7     node = input("Enter node: ")
8     m = int(input(f"Enter number of neighbours of {node}: "))
9     graph[node] = []
10
11     for _ in range(m):
12         nbr = input("Neighbour: ")
13         cost = int(input("Cost: "))
14         graph[node].append((nbr, cost))
15
16 h[node] = int(input(f"Heuristic value of {node}: "))
```

Enter region 1: A
Enter region 2: B
Enter region 3: C
Enter region 4: D
Enter neighbours of A (space-separated): B C
Enter neighbours of B (space-separated): A C D
Enter neighbours of C (space-separated): A B D
Enter neighbours of D (space-separated): B C

Color Assignment:
A -> Red
B -> Green
C -> Blue
D -> Red

Result

The map was colored successfully without assigning the same color to adjacent regions.

Conclusion

The Map Coloring Problem was solved successfully using the Constraint Satisfaction Problem (CSP) approach.